Relationship back_populates

Now you know how to use the **relationship attributes** to manipulate connected data in the database! 🎉

Let's now take a small step back and review how we defined those `Relationship()` attributes again, let's clarify that `back_populates` argument. 🤓

Relationship with `back_populates`

So, what is that `back_populates` argument in each `Relationship()`?

The value is a string with the name of the attribute in the **other** model class.

```
class Team(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    headquarters: str

    heroes: List["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: Optional[int] = None

    team_id: Optional[int] = Field(default=None, foreign_key="team.id")
    team: Optional[Team] = Relationship(back_populates="heroes")
```

That tells **SQLModel** that if something changes in this model, it should change that attribute in the other model, and it will work even before committing with the session (that would force a refresh of the data).

Let's understand that better with an example.

An Incomplete Relationship

Let's see how that works by writing an **incomplete** version first, without `back_populates`:

Python 3.10+

```
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
```

```
    headquarters: str

    heroes: list["Hero"] = Relationship()


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship()


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship()


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Read Data Objects

Now, we will get the **Spider-Boy** hero and, *independently*, the **Preventers** team using two `select`s.

As you already know how this works, I won't separate that in a select `statement`, `results`, etc. Let's use the shorter form in a single call:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship()


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship()


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Tip

When writing your own code, this is probably the style you will use most often, as it's shorter, more convenient, and you still get all the power of autocompletion and inline errors.

Print the Data

Now, let's print the current **Spider-Boy**, the current **Preventers** team, and particularly, the current **Preventers** list of heroes:

Python 3.10+

```
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship()


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship()


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Up to this point, it's all good. 😊

In particular, the result of printing `preventers_team.heroes` is:

```
Preventers Team Heroes: [
        Hero(name='Rusty-Man', age=48, id=2, secret_name='Tommy Sharp', team_id=2),
        Hero(name='Spider-Boy', age=None, id=3, secret_name='Pedro Parqueador', team_id=2),
        Hero(name='Tarantula', age=32, id=6, secret_name='Natalia Roman-on', team_id=2),
        Hero(name='Dr. Weird', age=36, id=7, secret_name='Steve Weird', team_id=2),
        Hero(name='Captain North America', age=93, id=8, secret_name='Esteban Rogelios',
team_id=2)
]
```

Notice that we have **Spider-Boy** there.

Update Objects Before Committing

Now let's update **Spider-Boy**, removing him from the team by setting `hero_spider_boy.team = None` and then let's print this object again:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship()


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship()


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

The first important thing is, we *haven't committed* the hero yet, so accessing the list of heroes would not trigger an automatic refresh.

But in our code, in this exact point in time, we already said that **Spider-Boy** is no longer part of the **Preventers**. 🔥

> **Tip**
>
> We could revert that later by not committing the **session**, but that's not what we are interested in here.

Here, at this point in the code, in memory, the code expects **Preventers** to *not include* **Spider-Boy**.

The output of printing `hero_spider_boy` without team is:

```
Spider-Boy without team: name='Spider-Boy' age=None id=3 secret_name='Pedro Parqueador' team_id=2
team=None
```

Cool, the team is set to `None`, the `team_id` attribute still has the team ID until we save it. But that's okay as we are now working mainly with the **relationship attributes** and the objects. ✅

But now, what happens when we print the `preventers_team.heroes`?

```
Preventers Team Heroes again: [
        Hero(name='Rusty-Man', age=48, id=2, secret_name='Tommy Sharp', team_id=2),
        Hero(name='Spider-Boy', age=None, id=3, secret_name='Pedro Parqueador', team_id=2,
team=None),
        Hero(name='Tarantula', age=32, id=6, secret_name='Natalia Roman-on', team_id=2),
        Hero(name='Dr. Weird', age=36, id=7, secret_name='Steve Weird', team_id=2),
        Hero(name='Captain North America', age=93, id=8, secret_name='Esteban Rogelios',
team_id=2)
]
```

Oh, no! 😱 **Spider-Boy** is still listed there!

Commit and Print

Now, if we commit it and print again:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship()


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship()


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

When we access `preventers_team.heroes` after the `commit`, that triggers a refresh, so we get the latest list, without **Spider-Boy**, so that's fine again:

```
INFO Engine SELECT hero.id AS hero_id, hero.name AS hero_name, hero.secret_name AS
hero_secret_name, hero.age AS hero_age, hero.team_id AS hero_team_id
FROM hero
WHERE ? = hero.team_id
2021-08-13 11:15:24,658 INFO sqlalchemy.engine.Engine [cached since 0.1924s ago] (2,)

Preventers Team Heroes after commit: [
        Hero(name='Rusty-Man', age=48, id=2, secret_name='Tommy Sharp', team_id=2),
        Hero(name='Tarantula', age=32, id=6, secret_name='Natalia Roman-on', team_id=2),
        Hero(name='Dr. Weird', age=36, id=7, secret_name='Steve Weird', team_id=2),
        Hero(name='Captain North America', age=93, id=8, secret_name='Esteban Rogelios',
team_id=2)
]
```

There's no **Spider-Boy** after committing, so that's good. 😊

But we still have that inconsistency in that previous point above.

If we use the objects before committing, we could end up having errors. 😔

Let's fix that. 🤓

Fix It Using `back_populates`

That's what `back_populates` is for. ✨

Let's add it back:

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

And we can keep the rest of the code the same:

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

> **Tip**
>
> This is the same section where we updated `hero_spider_boy.team` to `None` but we *haven't committed* that change yet.
>
> The same section that caused a problem before.

Review the Result

This time, **SQLModel** (actually SQLAlchemy) will be able to notice the change, and **automatically update the list of heroes** in the team, even before we commit.

That second print would output:

```
Preventers Team Heroes again: [
        Hero(name='Rusty-Man', age=48, id=2, secret_name='Tommy Sharp', team_id=2),
        Hero(name='Tarantula', age=32, id=6, secret_name='Natalia Roman-on', team_id=2),
        Hero(name='Dr. Weird', age=36, id=7, secret_name='Steve Weird', team_id=2),
        Hero(name='Captain North America', age=93, id=8, secret_name='Esteban Rogelios',
team_id=2)
]
```

Notice that now **Spider-Boy** is not there, we fixed it with `back_populates`! 🎉

The Value of `back_populates`

Now that you know why `back_populates` is there, let's review the exact value again.

It's quite simple code, it's just a string, but it might be confusing to think exactly *what* string should go there:

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

The string in `back_populates` is the name of the attribute *in the other* model, that will reference *the current* model.

```python
class Team(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    headquarters: str

    heroes: List["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: Optional[int] = None

    team_id: Optional[int] = Field(default=None, foreign_key="team.id")
    team: Optional[Team] = Relationship(back_populates="heroes")
```

So, in the class `Team`, we have an attribute `heroes` and we declare it with `Relationship(back_populates="team")`.

Python 3.10+

```python
# Code above omitted 👆

class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

The string in `back_populates="team"` refers to the attribute `team` in the class `Hero` (the other class).

And, in the class `Hero`, we declare an attribute `team`, and we declare it with `Relationship(back_populates="heroes")`.

So, the string `"heroes"` refers to the attribute `heroes` in the class `Team`.

Python 3.10+

```python
# Code above omitted 👆

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()

        preventers_team = session.exec(
            select(Team).where(Team.name == "Preventers")
        ).one()

        print("Hero Spider-Boy:", hero_spider_boy)
        print("Preventers Team:", preventers_team)
        print("Preventers Team Heroes:", preventers_team.heroes)

        hero_spider_boy.team = None

        print("Spider-Boy without team:", hero_spider_boy)

        print("Preventers Team Heroes again:", preventers_team.heroes)

        session.add(hero_spider_boy)
        session.commit()
        print("After committing")

        session.refresh(hero_spider_boy)
        print("Spider-Boy after commit:", hero_spider_boy)

        print("Preventers Team Heroes after commit:", preventers_team.heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

> **Tip**
>
> Each **relationship attribute** points to the other one, in the other model, using `back_populates`.

Although it's simple code, it can be confusing to think about 😵, because the same line has concepts related to both models in multiple places:

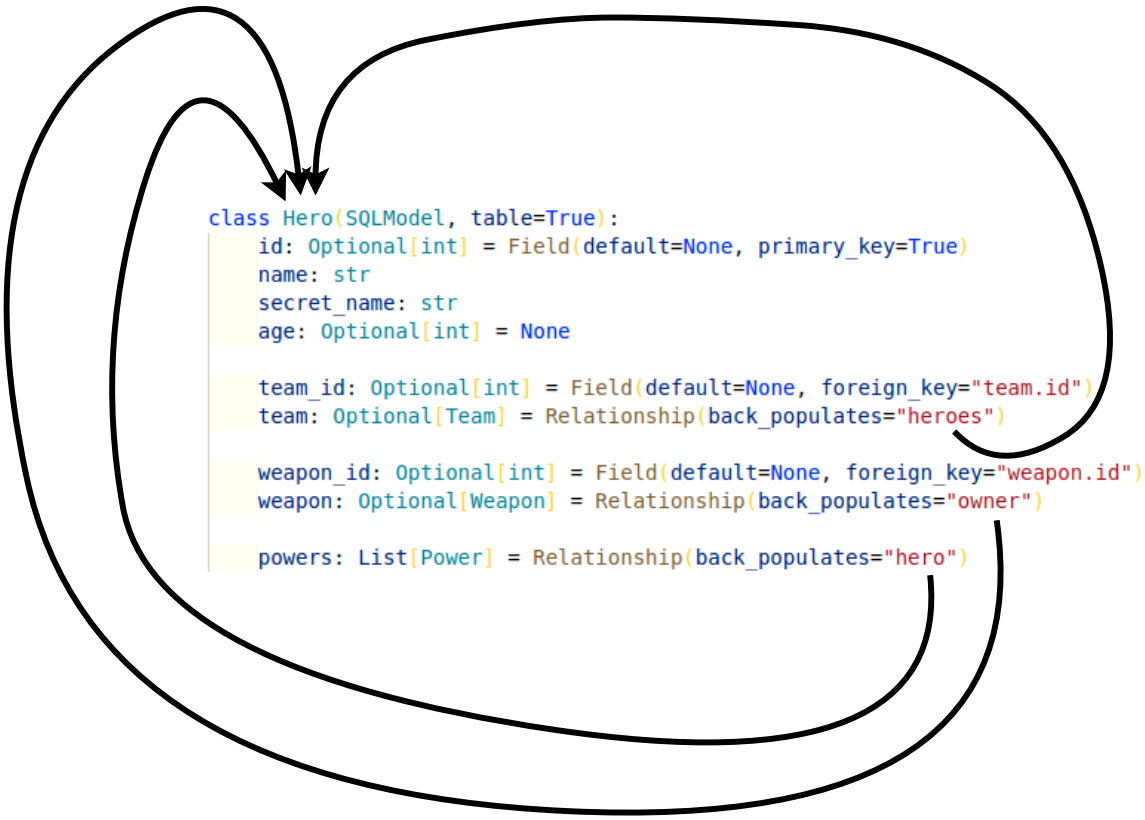
- Just by being in the **current** model, the line has something to do with the current model.
- The name of the attribute is about the **other** model.
- The type annotation is about the **other** model.
- And the `back_populates` refers to an attribute in the **other** model, that points to the **current** model.

A Mental Trick to Remember `back_populates`

A mental trick you can use to remember is that the string in `back_populates` is always about the current model class you are editing. 🤓

So, if you are in the class `Hero`, the value of `back_populates` for any relationship attribute connecting to **any** other table (to any other model, it could be `Team`, `Weapon`, `Powers`, etc) will still always refer to this same class.

So, `back_populates` would most probably be something like `"hero"` or `"heroes"`.



Python 3.10+

```python
# Code above omitted 👆

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
```

```python
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")

    weapon_id: int | None = Field(default=None, foreign_key="weapon.id")
    weapon: Weapon | None = Relationship(back_populates="hero")

    powers: list[Power] = Relationship(back_populates="hero")

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, SQLModel, create_engine


class Weapon(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)

    hero: "Hero" = Relationship(back_populates="weapon")


class Power(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)

    hero_id: int = Field(foreign_key="hero.id")
    hero: "Hero" = Relationship(back_populates="powers")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")

    weapon_id: int | None = Field(default=None, foreign_key="weapon.id")
    weapon: Weapon | None = Relationship(back_populates="hero")

    powers: list[Power] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def main():
    create_db_and_tables()


if __name__ == "__main__":
    main()
```